

hw2

November 22, 2025

1 ChatGPT questions

conversation here

1. it correctly identifies the incorrect implication direction so this seems right.
2. im not sure but i think its wrong on at least the second one since usually this diagonalization argument is between only DTIME classes. i think 3 is also traditionally between just DTIME classes
3. i think it gets the correct argument basically saying you cant check if the variables remain the same across all clauses
4. i think it is correct. i think the part showing np is in this class is trying to simulate the np program using the config graph, but instead of storing memory it queries the bit every time and it can remember where it was on the certificate and reread. it allows checks for global consistency

2 regular questions

1. 4.2.2 Prove that 2SAT is NL-complete.

This proof proves that 2SAT is in co-NL and by NL = co-NL 2SAT is in NL. construct a certificate by making a graph encoding a node for each variable and its negation. then connect two nodes a, b if $\neg a \implies b$. to check for validity, check over all (up to poly number of variables) variables x for paths from x to x' where x' is the negation of x and a path from x' back to x . if there exists a clause where such a path exists, then the 2 sat instance must be unsatisfiable since the existence of a satisfying assignment implies contradiction $x \wedge \neg x$. if there is no contradiction, then this implies there exists an assignment that satisfies all clauses.

now let us reduce DPATH to $\overline{2SAT}$. start with a clause $x_s \vee x_s$. this represents the starting vertex. for each edge from $u \rightarrow v$, add a clause $(\neg x_u \vee x_v)$. finally, add a clause $\neg x_t \vee \neg x_t$. if there exists a path, then the 2sat instance is not satisfiable since there would exist a chain of implications requiring x_t to be true, but the final clause requires x_t to be

false. if there isnt a path, then the clause is satisfiable since none of the restrictions cause contradiction except for the constructed chains of implication; satisfy the implications required by x_s they run out, then satisfy the implications required by $\neg x_t$. since there is no path, they do not interact.

this reduction is logspace since one can construct the 2sat by enumerating over all the nodes, then locally constructing clauses over all edges of that node. (repeats are fine)

i used wikipedia for the sketch and chat gpt to check for correctness

2. 3.2.1

Prove that: if $\text{PSPACE} \neq \text{EXP}$, then $\mathbb{L} \neq \mathbb{P}$.

we will prove the contrapositive, i.e. we want to prove if $\mathbb{P} \subseteq \mathbb{L}$, then $\text{EXP} \subseteq \text{PSPACE}$

consider a DTM M_E that decides if a string $A \in \mathcal{L}$ in exponential time wrt the length of the input. suppose the turing machine M decides in $2^{|n|^k}$, $n = |A|$ time for some constant k .

if we pad the string A into A_{pad} , then there exists a polynomial turing machine that takes in the input A_{pad} and decides if $A \in \mathcal{L}$ in $2^{|n|^k}$ steps. since $\mathbb{L} = \mathbb{P}$, there exists a turing machine M_l that decides if padded string A_{pad} is in $\mathcal{L}$ with log space.

M_l decides if $A \in \mathcal{L}$ in $O(\log x)$ where x is the size of the input, then M_l decides terminates in $O(\log(2^{|n|^k})) = O(|n|^k)$ size. thus if a machine M' can run M_l with input A_{pad} , then M' would be in PSPACE .

we can simulate M_l with poly space by storing the input A as is, then when M_l requests for a character outside the length of $|A|$, return the padding character. the space required to run M_l is also polynomial, thus we have constructed an M' for any M_E that decides the same language as M_E in PSPACE